

Experiment 4

Arman Imam

EEE3342C

25Fall 0011

Equipment:

Computer with Xilinx ISE software, Vivado 2017.4 specifically.

Objective:

The objective of experiment 4 is to introduce the user to combination logics, with one of the various ways to add two binary numbers together. For part 1, the user is asked to create a half adder and full adder and test it in the test bench. The full adder should be using the half adder module to reference. For part 2, The user is asked to create an 8-bit Ripple Carry Adder logic in the source code, and to test it using two 8 bit binary numbers. For part 3, the user is asked to create an 8-bit Carry-Look-Ahead logic in the source code, and to test it using two 8 bit binary numbers. In part 4, the user is asked to create a mode select between 32-bit Adder and Subtractor. What this entails is that when Cin is 0, the logic acts as a adder, and when Cin is 1, the logic acts as a subtractor.

Design Steps:

I began by creating 4 separate folders that account for parts 1-4. For part 1, I created a half adder using inputs A, B, and outputs Sum, Count, where $\text{Sum} = A \oplus B$ and $\text{Count} = A \& B$. I also create a full adder, which was instantiated by the half adder, with a new input called Cin. For Part 2, I created an 8-bit Ripple Carry adder, where the inputs A, B, and output Sum are 8 bits. I instantiated the Full adder for each iteration by using a generate command. For part 3, I did the same thing, but I created a Carry-Over-Ahead Adder. For part 4, I created a 32-bit mode select between an Adder and Subtractor. I created a 32 bit Ripple Carry Adder to instantiate in the mode select code.

Part 1:

Verilog RTL of Half Adder

The source code for the half adder has two inputs(A, B) and 2 outputs(Sum, Count). Sum is equal to $A \oplus B$ and Count is equal to $A \& B$. The sum represents the sum of two bits, and count represents as the carryover, if $A + B$ results in a 2 bit result.

```
1 | `timescale 1ns / 1ps
2 |
3 | //Half Adder
4 | module Half_Adder(
5 |     input A, B,
6 |     output Sum, Count
7 | );
8 |     assign Sum = A ^ B;
9 |     assign Count = A & B;
10| endmodule
```

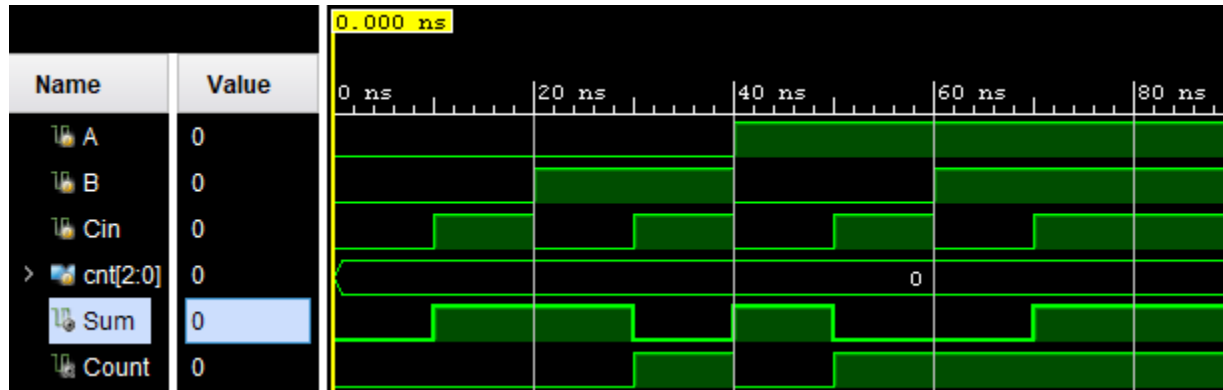
Verilog RTL of Full Adder

The source code for the full adder has three inputs(A, B, Cin) and 2 outputs(Sum, Count). In this particular source code, I've instantiated the half adder to create the full adder. Sum is essentially equal to $A \oplus B \oplus Cin$ and Count is essentially equal to $(A \& B) + (Cin \& (A \oplus B))$. The sum represents the sum of two bits, and count represents as the carryover, if $A + B + Cin$ results in a 3 bit result.

```
1 | `timescale 1ns / 1ps
2 |
3 | //Full Adder
4 | module Full_Adder(
5 |     input A, B, Cin,
6 |     output Sum, Count
7 | );
8 |
9 |     wire sum0, count0, sum1, count1;
10|    // Sum = A ^ B ^ Cin
11|    //Count = (A&B) & (B&C) & (Cin&A)
12|    Half_Adder UUT(.A(A), .B(B), .Sum(sum0), .Count(count0));
13|    Half_Adder AAT(.A(Cin), .B(sum0), .Sum(Sum), .Count(count1));
14|
15|    assign Count = count0 | count1;
16|
17|
18| endmodule
```

Screenshot of 1-bit Adder waveform of 1'b1 + 1'b1 and 1'b1 + 1'b0

For every iteration of A, B, and Cin, it reflects how adding the 3 inputs together influences the sum and count outputs. For example, when (A,B,Cin order) 110 is inputted, the sum will be 0, but the count will be 1 since it creates a 2bit result.



Part 2:

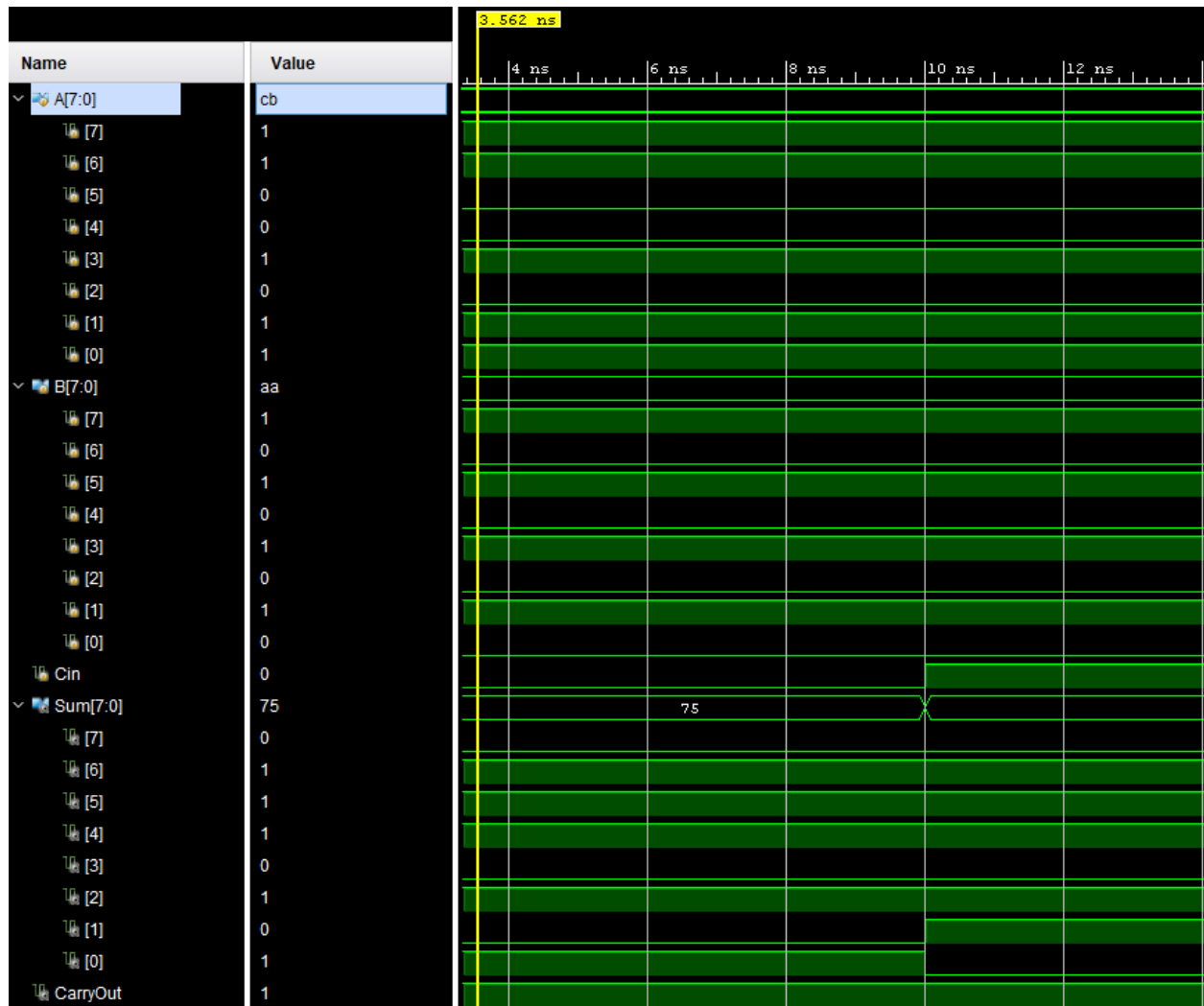
Verilog RTL of 8-bit RCA

The source code for the 8 bit ripple carry adder has inputs(8 bit A and B, 1 bit Cin), and outputs (8 bit Sum, 1 bit Carryout). This combination logic revolves around full adders using other full adders in sequential order. For an 8 bit RCA, I used the generate command and for loop to create 8 different Full Adders, where the Count for one adder is the same as the Cin for the next adder. The CarryOut output would be the last Count that's be iterated, which would be Carry[8] in this case.

```
1  `timescale 1ns / 1ps
2
3  // Ripple Carry Adder (RCA)
4  module RCA(
5      input [7:0] A, B,
6      input Cin,
7      output [7:0] Sum,
8      output CarryOut
9  );
10
11     genvar i;
12     wire[8:0] Carry;
13     assign Carry[0] = Cin;
14
15     generate
16         // n is the number of blocks youd like to make
17         for(i = 0; i < 8; i = i + 1)begin
18             Full_Adder ABT(
19                 .A(A[i]),
20                 .B(B[i]),
21                 .Cin(Carry[i]),
22                 .Sum(Sum[i]),
23                 .Count(Carry[i+1])
24             );
25         end
26     endgenerate
27     assign CarryOut = Carry[8];
28 endmodule
```

Screenshot of waveform of 8-bit RCA: 8'b11001011 + 8'b10101010

I've tested the source code by setting A to 8'b11001011 and B to 8'b10101010. When Cin is 0, the sum in hexadecimal is 75. When Cin is 1, the sum is 76 in hexadecimal. The CarryOut(or count) is 1 for both iterations. These results are expected.



Provide the binary and decimal value of the addition

Printed in TCL Console. Since there is a carry over, the sum of A, B, and C isn't the true value in decimal form. If we were using a 9 bit result, Cin=0 would have the sum 373 and 374 if Cin=1.

```
A=11001011 (203) B=10101010 (170) Cin=0 | Sum=01110101 (117) CarryOut=1
A=11001011 (203) B=10101010 (170) Cin=1 | Sum=01110110 (118) CarryOut=1
```

Part 3:

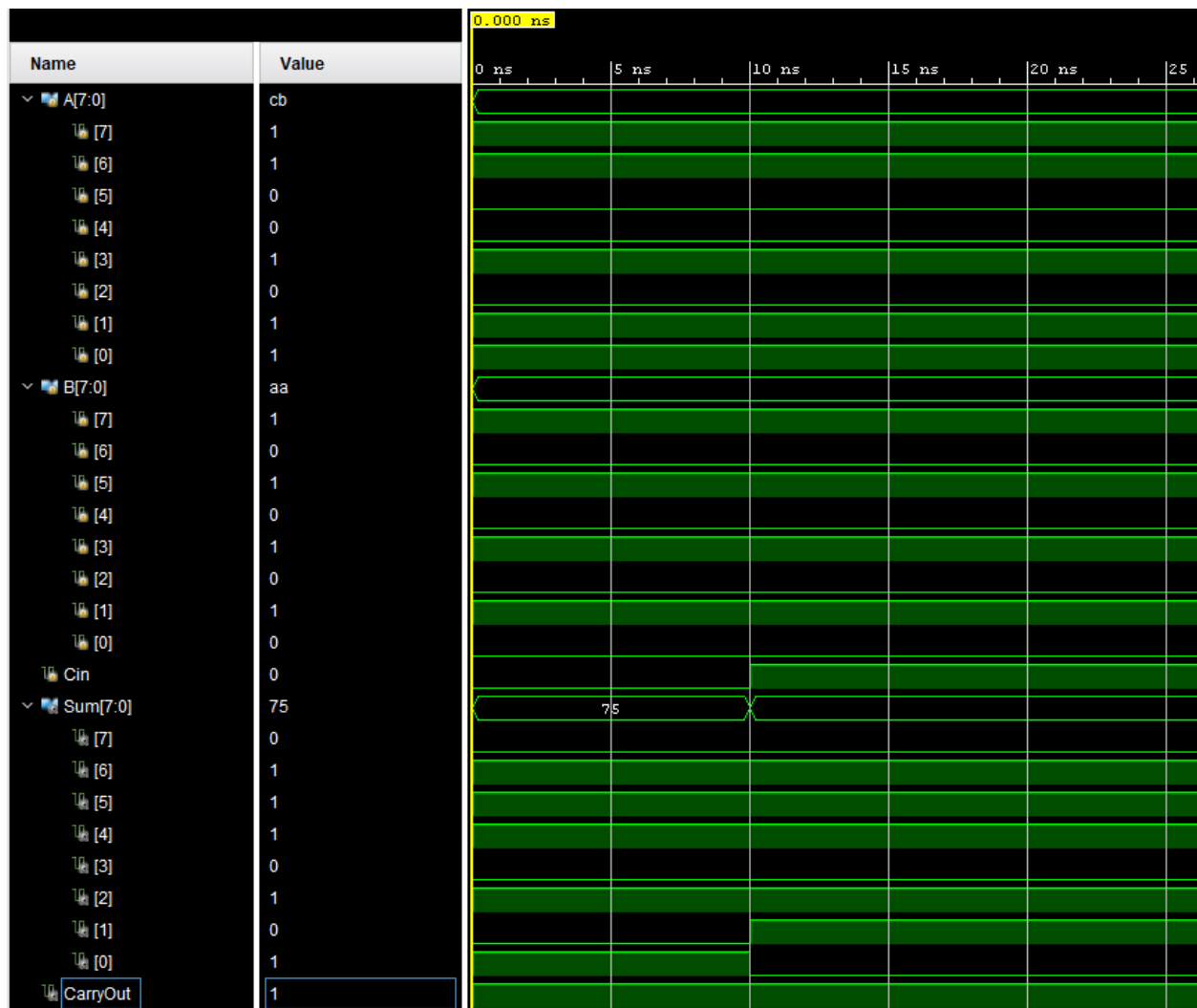
Verilog RTL of 8-bit CLA

The source code for the 8 bit carry look ahead adder possesses the same inputs and outputs as the RCA, which include inputs(8 bit A and B, 1 bit Cin), and outputs (8 bit Sum, 1 bit Carryout). This combination logic relies on full adders that are processed in parallel into the carry look ahead logic. I established a carry-generate (G), which is $A \text{ XOR } B$, and a carry-propagate (P), which is $A \text{ AND } B$, for every full adder. I created a C wire that reflects on the design $C(n+1) = G_n + P_n * C_n$, where $C[0]$ is Cin. For the equation, " $S = P \wedge C_{in}$ ", I didn't include it in the code because the same logic was being executed by the Full Adder, since P was already $A \wedge B$. By default, the sum of a full adder is $A \wedge B \wedge C_{in}$.

```
1 | `timescale 1ns / 1ps
2 | //Carry Look Ahead Adder (CLA)
3 | module CLA(
4 |     input[7:0] A, B,
5 |     input Cin,
6 |     output[7:0] Sum,
7 |     output CarryOut
8 | );
9 | //Carry-propagate (P) and Carry-generate (G)
10 | wire[7:0] P, G, Carry;
11 | wire[8:0] C;
12 | assign C[0] = Cin;
13 | // Generate P, G, C, and the Full Adder
14 | genvar i;
15 | generate
16 |     for (i=0; i<8; i=i+1) begin
17 |         assign P[i] = A[i] ^ B[i];
18 |         assign G[i] = A[i] & B[i];
19 |         assign C[i+1] = G[i] | (P[i] & C[i]);
20 |         //assign Sum[i] = P[i] ^ C[i]; This code is already being executed by the full adder sum logic
21 |         // Sum[i] = (A[i] ^ B[i]) ^ C[i]
22 |         Full_Adder ACT(
23 |             .A(A[i]),
24 |             .B(B[i]),
25 |             .Cin(C[i]),
26 |             .Sum(Sum[i]),
27 |             .Count(Carry[i])
28 |         );
29 |     end
30 | endgenerate
31 | assign CarryOut = C[8];
32 | endmodule
```

Screenshot of waveform of 8-bit CLA: $8'b11001011 + 8'b10101010$

The results of CLA are the exact same as RCA, which proves that both adders perform the same tasks.



Verify that the RCA and CLA produce the same output

For both RCA and CLA waveforms, the Sum and Count produce the same outputs for when Cin is 0 and 1. For Cin = 0, sum is 01110101 and Count is 1. For Cin = 1, sum is 01110110 and Count is 1.

Part 4:

Verilog RTL of 32-bit Add/Sub Module

In the source code, for the 32-bit Add/Sub module, I used inputs(32 bit A and B, 1 bit mode) and outputs (32 bit results, and 1 bit Cout). My goal was to make it so when mode=0, it's an adder, and when mode=1, it is a subtractor. To do that, I needed to make a wire that takes one of the inputs(A, B) to XOR the mode; so I made $Bor = B[i] \wedge mode$, for 32 different iterations. For this particular module, I created a 32 bit RCA, as it was easy to make changes to the 8 bit RCA I've previously written. This way, I'm able to simplify the code and rely less on for loops, which can create some confusion.

```
1  `timescale 1ns / 1ps
2  //Addition/Subtraction module
3  module ModeSelect(
4      input[31:0] A, B,
5      input mode,
6      output[31:0] Results,
7      output Cout
8  );
9      genvar i;
10     //values determined by mode.
11     wire [31:0] Bor;
12     generate
13         for(i = 0; i<32; i = i+1)begin
14             assign Bor[i] = B[i] ^ mode;
15         end
16     endgenerate
17
18     RCA_32bit ATS(
19         .A(A),
20         .B(Bor),
21         .Cin(mode),
22         .Sum(Results),
23         .CarryOut(Cout)
24     );
25
26 endmodule
```

32 bit RCA:

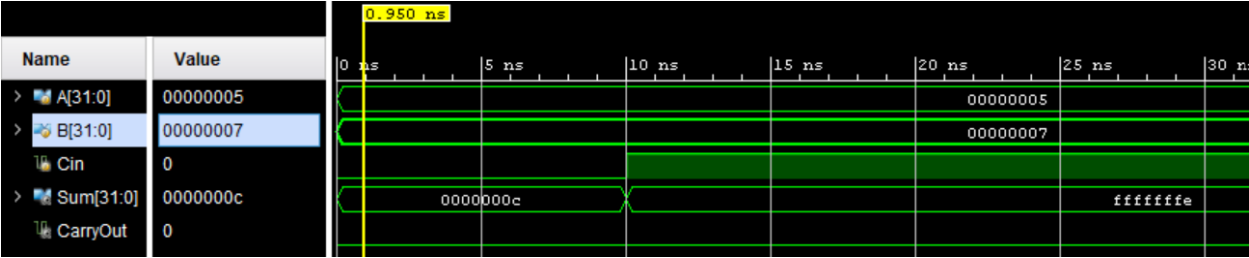
```
3 // Ripple Carry Adder in 32 bit (RCA)
4 module RCA_32bit(
5     input [31:0] A, B,
6     input Cin,
7     output [31:0] Sum,
8     output CarryOut
9 );
10
11     genvar i;
12     wire[32:0] Carry;
13     assign Carry[0] = Cin;
14
15     generate
16         // n is the number of blocks youd like to make
17         for(i = 0; i < 32; i = i + 1)begin
18             Full_Adder ABT(
19                 .A(A[i]),
20                 .B(B[i]),
21                 .Cin(Carry[i]),
22                 .Sum(Sum[i]),
23                 .Count(Carry[i+1])
24             );
25         end
26     endgenerate
27     assign CarryOut = Carry[32];
28 endmodule
29
```

Truth table relationship between “Mode” signal and chosen input bit signal

mode	Bor[i]	$Bor[i] = B[i] \wedge mode$	Operation
0	0	0	Addition
0	1	1	Addition
1	0	1	Subtraction
1	1	0	Subtraction

Screenshot of waveform of an addition mode and subtraction mode

The waveform shows the expected result. The sum is given in hexadecimal, which is essentially 12 in decimal. For when Cin = 1, the hexadecimal is essentially a -2, which is in 2's complement.



Q&A:

How many gates are required to make a 1-bit FAs?

5 gates are needed: 2 XOR, 2 AND, and 1 OR.

What are the four tradeoffs that computer hardware engineers need to consider when designing any digital system?

Power Consumption, Speed, Complexity, and Resources

What are the pros and cons of an RCA?

Pros of Ripple Carry Adder: simple design, uses as few gates possible, and can easily be adjusted for higher bits.

Cons of Ripple Carry Adder: Slower performance due to sequential order.

What are the pros and cons of a CLA?

Pros of Carry-Look-Ahead Adder: uses parallel design structure for increased speed of the program, Higher efficiency

Cons of Carry-Look-Ahead Adder: More complex design and logic, higher power consumption

How many gates were used in the CLA versus the RCA in Part 2 and 3?

RCA used 8 adders, which each had 2 XOR, 2 AND, and 1 OR. In total, has 40 gates

CLA used 8 XOR gates for P, 8 AND gates for G, 8 OR and 8 AND for $G[i] \mid (P[i] \& C[i])$. 8 adders were also used, with 8 gates each. 72 Gates in total
RCA is far more efficient when it comes to using gates.

Test Plan:

Part1_Half Adder: For this test plan, I went through every iteration for the Half Adder. Here I've instantiated the Half adder module, and tested the values.

```

1  `timescale 1ns / 1ps
2
3  module adder_sim();
4
5
6  parameter NUM_INP =2;
7  reg A,B;
8  reg[NUM_INP-1:0] cnt;
9  wire Sum, Count;
10 integer i;
11
12 Half_Adder AUT(.A(A), .B(B), .Sum(Sum), .Count(Count));
13 initial begin
14     cnt = 0;
15     for(i = 0; i < 2**NUM_INP; i = i + 1)begin
16         {A,B} = i;
17         #10;
18     end
19
20 end
21 endmodule
22

```

Part1_Full Adder: For this test plan, I went through every iteration for the Full Adder. Here I've instantiated the Full adder module, and tested the values.

```

1  `timescale 1ns / 1ps
2
3  module adder_sim();
4
5
6  parameter NUM_INP =3;
7  reg A,B,Cin;
8  reg[NUM_INP-1:0] cnt;
9  wire Sum, Count;
10 integer i;
11
12 Full_Adder AUT(.A(A), .B(B), .Cin(Cin), .Sum(Sum), .Count(Count));
13 initial begin
14     cnt = 0;
15     for(i = 0; i < 2**NUM_INP; i = i + 1)begin
16         {A,B,Cin} = i;
17         #10;
18     end
19
20 end
21 endmodule

```

Part2: In this test bench, I've let A = 8'b11001011 and B = 8'b10101010, for when Cin = 0 and Cin = 1. I've also included the monitor command to show that the values were as expected. Here I've instantiated the RCA module and tested the values.

```
1  `timescale 1ns / 1ps
2  module adder_sim();
3
4  reg [7:0] A, B;
5  reg Cin;
6  wire [7:0] Sum;
7  wire CarryOut;
8
9  RCA AUT(.A(A), .B(B), .Cin(Cin), .Sum(Sum), .CarryOut(CarryOut));
10 initial begin
11     $monitor("A=%b (%0d) B=%b (%0d) Cin=%b | Sum=%b (%0d) CarryOut=%b",
12             A, A,
13             B, B,
14             Cin,
15             Sum, Sum,
16             CarryOut);
17 end
18
19 initial begin
20     A = 8'b11001011;
21     B = 8'b10101010;
22     Cin = 1'b0;
23     #1;
24     #10;
25     A = 8'b11001011;
26     B = 8'b10101010;
27     Cin = 1'b1;
28     #1;
29     #10;
30 end
```

Part3: In this test bench, I've used the same test bench as part 4, but I instantiated the module CLA instead of RCA.

```

1  `timescale 1ns / 1ps
2  module adder_sim();
3
4      reg [7:0] A, B;
5      reg Cin;
6      wire [7:0] Sum;
7      wire CarryOut;
8
9      CLA AST(.A(A), .B(B), .Cin(Cin), .Sum(Sum), .CarryOut(CarryOut));
10 initial begin
11     A = 8'b11001011;
12     B = 8'b10101010;
13     Cin = 1'b0;
14     #10;
15     A = 8'b11001011;
16     B = 8'b10101010;
17     Cin = 1'b1;
18     #10;
19 end
20
21 endmodule

```

Part4: In this test bench, I've instantiated the ModeSelect module. The values I've chosen are essentially $5 + 7$ or $5 - 7$ in decimal depending on if mode = 0 (adding) or if mode = 1(subtracting).

```

1  `timescale 1ns / 1ps
2  module adder_sim();
3
4      reg [31:0] A, B;
5      reg Cin;
6      wire [31:0] Sum;
7      wire CarryOut;
8
9      ModeSelect AST(.A(A), .B(B), .mode(Cin), .Results(Sum), .Cout(CarryOut));
10 initial begin
11     A = 32'b00000000000000000000000000000101;
12     B = 32'b00000000000000000000000000000111;
13     Cin = 1'b0;
14     #10;
15     A = 32'b00000000000000000000000000000101;
16     B = 32'b00000000000000000000000000000111;
17     Cin = 1'b1;
18     #10;
19 end
20
21 endmodule

```

Conclusion

This experiment was a success. The results were as expected for all parts of the experiment. I've hit a few roadblocks with parts 2-4, due to how inexperienced and how little I understood the assignment initially. I didn't understand how to reflect the logic diagrams into code, until I played around with the software throughout the week, that I understood how to do it. Testing combinational logics using this software has furthered my understanding of such logic, and how versatile it can be.